

Funções, Procedimentos e Passagem de Parâmetros por Valor e por Referência em Linguagem C

Professor: Vinícius Breda Silva



Objetivos da aula

Entender por que dividimos um programa em partes menores.

Diferenciar procedimento de função.

Usar parâmetros para enviar dados para uma função.

Entender passagem por valor e por referência.

Praticar com exercícios progressivos em linguagem C.

Por que usar funções?

Organização

Cada parte do programa fica responsável por uma tarefa.

Reutilização

Escrevemos uma lógica uma vez e podemos chamá-la várias vezes.

Manutenção

Fica mais fácil encontrar erros e alterar o programa.

Uma tarefa isolada

```
void exibirMensagem()  
{  
    printf("Ola, turma!");  
}
```

O programa principal chama a rotina quando precisar.

Pense em uma função como uma pequena “máquina” com uma responsabilidade.

Procedimento: executa uma ação

Procedimento é uma rotina que executa uma tarefa, mas não devolve um valor ao ponto que a chamou.

Em C, usamos uma função com retorno void.

Pode receber parâmetros ou não.

VOID

void = a rotina *realiza algo*, mas *não entrega* um valor de volta.

```
void saudacao()  
{  
    printf("Bem-vindo!");  
}  
  
void main()  
{  
    saudacao();  
}
```

Função: calcula e devolve um valor

Uma função pode **receber dados**, processá-los e **retornar** um resultado.

O **tipo** antes do nome indica o tipo do valor retornado.

A palavra **return** devolve o resultado.

```
int somar(int a, int b)
{
    int resultado;
    resultado = a + b;
    return resultado;
}

void main()
{
    int total;
    total = somar(10, 5);
}
```

Parâmetros: dados que entram na rotina

Parâmetros

Variáveis declaradas na definição da função.

Ex.: int a, int b

Argumentos

Valores enviados na chamada.

Ex.: somar(10, 5)

Resultado

A rotina usa os valores recebidos para executar sua tarefa.

```
int somar(int a, int b)
{
    return a + b;
}

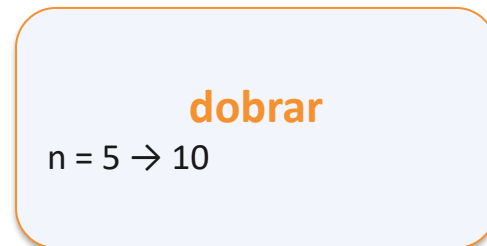
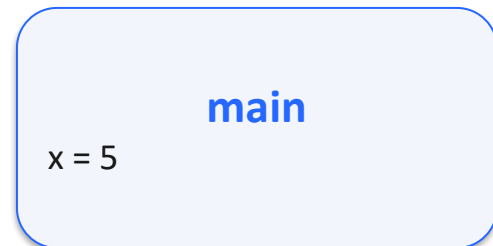
resultado = somar(10, 5);
```

Passagem por valor

A função recebe uma **CÓPIA** do valor da variável.

Alterar o parâmetro dentro da função não altera a variável original.

É o comportamento padrão para tipos simples em C.



```
void dobrar(int n)
{
    n = n * 2;
}

void main()
{
    int x = 5;
    dobrar(x);
    printf("%d", x); // 5
}
```

x continua valendo 5

Passagem por referência

Queremos que a rotina consiga alterar a variável original.

Em C, fazemos isso passando o endereço da variável com **&**.

O parâmetro recebe um ponteiro *****, que permite acessar o conteúdo naquele endereço.

main

x = 5 → 10

dobrar

n aponta para x

```
void dobrar(int *n)
{
    *n = *n * 2;
}

void main()
{
    int x = 5;
    dobrar(&x);
    printf("%d", x); // 10
}
```

Agora a rotina altera o próprio x

Valor × Referência

POR VALOR

- Recebe uma cópia
- Usa o valor normalmente: `n`
- Original não é alterado
- Chamada: `rotina(x)`

POR REFERÊNCIA

- Recebe o endereço
- Parâmetro usa ponteiro: `*n`
- Pode alterar o original
- Chamada: `rotina(&x)`

Exemplo 1 — Procedimento com parâmetros por valor

```
void exibirMaior(int a, int b)
{
    if (a > b)
        printf("Maior: %d", a);
    else
        printf("Maior: %d", b);
}

void main()
{
    int n1, n2;
    scanf("%d", &n1);
    scanf("%d", &n2);
    exibirMaior(n1, n2);
}
```

1. Quais são os parâmetros?
2. O procedimento retorna algo?
3. n1 e n2 podem ser alterados por essa rotina?

Exemplo 2 — Função com passagem de parâmetros por valor e com retorno

```
float calcularMedia(float n1, float n2)
{
    return (n1 + n2) / 2;
}

void main()
{
    float nota1, nota2, media;

    scanf("%f", &nota1);
    scanf("%f", &nota2);

    media = calcularMedia(nota1, nota2);
    printf("Media: %.2f", media);
}
```

Entrada: **duas notas**

Processamento: cálculo da média

Saída da função: um float representando o **resultado** da operação

A função não precisa alterar as notas originais → passagem por valor.

Exemplo 3 — Procedimento com passagem de parâmetros por referência

```
void trocar(int *a, int *b)
{
    int aux;
    aux = *a;
    *a = *b;
    *b = aux;
}

void main()
{
    int x = 10, y = 20;
    trocar(&x, &y);
    printf("x=%d y=%d", x, y);
}
```

Antes: x = 10 e y = 20

A rotina recebe os endereços de x e y.

Depois: x = 20 e y = 10

Excelente exemplo para visualizar por que referência é necessária.

O printf exibirá: x = 20 y = 10

Exercícios para treino

Do mais simples ao mais completo

1. Procedimento saudacao(): exibir uma mensagem na tela.
2. Função somar(a, b): receber dois inteiros e retornar a soma.
3. Função maior(a, b): retornar o maior entre dois números.
4. Função calcularMedia(n1, n2): retornar a média e, no main, informar aprovado/reprovado.
5. Procedimento dobrar(int *n): alterar por referência o valor informado.
6. Procedimento trocar(int *a, int *b): inverter dois valores.
7. Desafio: procedimento calcular(int a, int b, int *soma, int *subtracao) que devolva dois resultados por referência.

Desafio final — Mini Calculadora

Crie um programa que leia dois números e tenha rotinas separadas para: **somar**, **subtrair**, **multiplicar** e **dividir**.

As operações devem receber os números por **parâmetros**.

As funções devem **retornar o resultado** para o programa principal.

Depois, evolua: crie um procedimento que receba **referências** e devolva soma e subtração de uma única vez.

```
float somar(float a, float b);  
float subtrair(float a, float b);  
float multiplicar(float a, float b);  
float dividir(float a, float b);  
  
void main()  
{  
    // leitura  
    // chamadas das funções  
    // exibição dos resultados  
}
```

Resumo da aula

Procedimento

Executa uma tarefa e usa void.

Função

Executa uma tarefa e pode retornar um valor.

Parâmetros

Permitem enviar informações para as rotinas.

Por valor

A rotina recebe uma cópia.

Por referência

A rotina acessa a variável original.